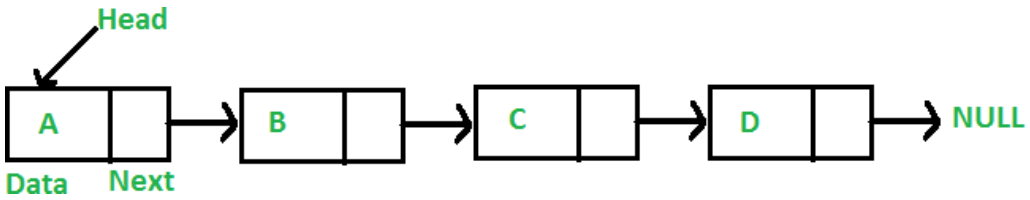
Unit 5. Data Structure Implementation and Applet Classes

Linked List

Like arrays, Linked List is a linear data structure. Unlike arrays, linked list elements are not stored at a contiguous location; the elements are linked usingpointers.



Why Linked List?

Arrays can be used to store linear data of similar types, but arrays have the following limitations.

1) The size of the arrays is fixed: So we must know the upper limit on the number of elements in advance. Also, generally, the allocated memory is equalto the upper limit irrespective of the usage.

2) Inserting a **new element** in an array of elements is expensive because the room has to be created for the new elements and to create room existing elements have to be shifted but in Linked list if we have the **head node(first node)** then wecan traverse to any node through it and **insert new node** at the required **position.**
For example, in a system, if we maintain a sorted list of IDs in an array id[].
id[] = [1000, 1010, 1050, 2000, 2040].

And if we want to insert a new ID 1005, then to maintain the sorted order, wehave to move all the elements after 1000 (excluding 1000).

Deletion is also expensive with arrays until unless some special techniques areused. For example, to delete 1010 in id[], everything after 1010 has to be moved due to this so much work is being done which affects the efficiency of the code.

Advantages over arrays

1) Dynamic size
2) Ease of insertion/deletion

Drawbacks:

1) Random access is not allowed. We have to access elements **sequentially starting from the first node(head node).** So we cannot do binary search withlinked lists

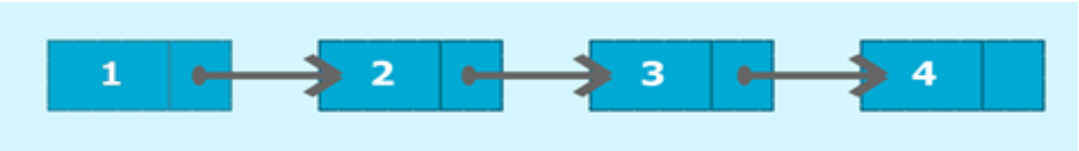
efficiently with its default implementation.
2) Extra memory **space for a pointer** is required with each element of the list.
3) Not store friendly. Since **array elements are contiguous** locations, there islocality of reference which **is not** there in case of linked lists.

Representation:
A linked list is represented by a pointer to the first node of the linked list. The first node is called the head. If the linked list is empty, then the value of the head points to NULL.
Each node in a list consists of at least two parts:
1) data (we can store integer, strings or any type of data).
2) Pointer (Or Reference) to the next node (connects one node to another)
In Java , LinkedList can be represented as a class and a Node as a separateclass.
The LinkedList class contains a reference of Node class type.

Java Program to create and display a singly linked list

The singly linked list is a linear data structure in which each element of the list contains a pointer which points to the next element in the list. **Each element** in the singly linked list is called a **node**.

Each node has two components: data and a pointer next
which points to the next node in the list. **The first node of the list is called as head, and the last node of the list is called a tail**. The last node of thelist contains a **pointer to the null.** Each node in the list can be accessed linearly by traversing through the list from head to tail.



Consider the above example; node 1 is the head of the list and node 4 is the tailof the list. Each node is connected in such a way that node 1 is pointing to node2 which in turn pointing to node 3. Node 3 is again pointing to node 4. Node 4 is pointing to null as it is the last node of the list.

Algorithm

1. Create a class Node which has two attributes: data and next. Next is apointer to the next node.
2. Create another class which has two attributes: head and tail.
   a. addNode() will add a new node to the list:

1. Create a new node.
2. It first checks, whether the head is equal to null which means thelist is empty.
3. If the list is empty, both head and tail will point to the newly addednode.
4. If the list is not empty, the new node will be added to end of the listsuch that tail's next will point to the newly added node. This new node will become the new tail of the list.

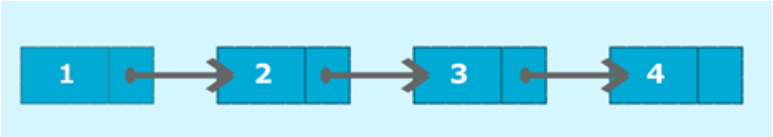
b. display() will display the nodes present in the list:

1. Define a node current which initially points to the head of the list.
2. Traverse through the list till current points to null.
3. Display each node by making current to point to node next to it in each iteration.

Java program to create a singly linked list of n nodes and count the number of nodes

In this program, we need to create a singly linked list and count the nodes present inthe list.

To accomplish this task, traverse through the list using node current which initially points to head. Increment current in such a way that current will point to its next node in each iteration and increment variable count by 1. In the end, the count will hold the value which denotes the number of nodes present in the list.



Algorithm

- Create a class Node which has two attributes: data and next. Next is apointer to the next node.
- Create another class which has two attributes: head and tail.

a. addNode() will add a new node to the list:

1. Create a new node.
2. It first checks, whether the head is equal to null which means thelist is empty.
3. If the list is empty, both head and tail will point to the newly addednode.
4. If the list is not empty, the new node will be added to end of the listsuch that tail's next will point to the newly added node. This new node will become the new tail of the list.

b. display() will display the nodes present in the list:

1. Define a node current which initially points to the head of the list.

2. Traverse through the list till **current points to null.**
3. Display each node by making current to point to node next to it in eachiteration.

C. countNodes() will count the nodes present in the list:

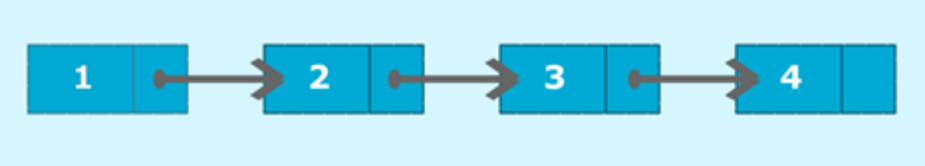
1. Define a node current which will initially point to the head of the list.
2. Declare and initialize a variable count to 0.
3. Traverse through the list till current point to null.
4. Increment the value of count by 1 for each node encountered in the list.

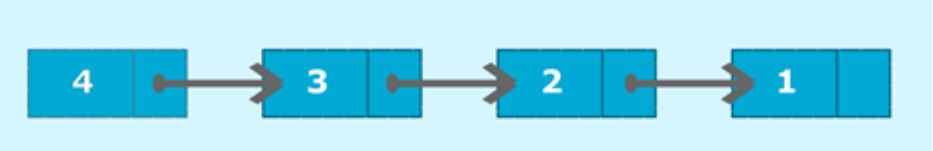
Java program to create a singly linked list of n nodesand display it in reverse order

In this program, we need to create a singly linked list and display the list in reverse order.

Original List

Reversed List





One of the approaches to solving this problem is to reach the end the of the list and display the nodes from tail to head recursively.

Algorithm

- Create a class Node which has two attributes: data and next. Next is apointer to the next node.
- Create another class which has two attributes: head and tail.

a. addNode() will add a new node to the list:

1. Create a new node.
2. It first checks, whether the head is equal to null which means thelist is empty.
3. If the **list is empty,** both **head and tail** will point to the **newly addednode.**
4. If the list is not empty, the new node will be **added to end** of the listsuch that **tail's next** will point to the **newly added node**. This new node will become

the new tail of the list.

b. display() will display the nodes present in the list:

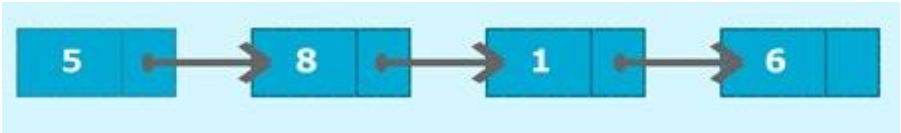
1. Define a node current which **initially points to the head** of the list.
2. Traverse through the list till **current points to null.**
3. Display each node by making current to point to node next to it in eachiteration.

C. reverse() will reverse the order of the nodes present in the list.

- This method checks whether node next to current is null which implies that, current is pointing to tail, then it will print the data of the tail node.
- Recursively call reverse() by considering node next to current node and prints out allthe nodes in reverse order starting from the tail.

Java program to find the maximum and minimumvalue node from a linked list

In this program, we need to find out the minimum and maximum value node in the given singly linked list.



We will maintain two variables min and max. Min will hold minimum value node, and max will hold maximum value node. In the above example, 1 will be minimum value node and 8 will be maximum value node. The algorithm to find the maximum and the minimum node is given below.

Algorithm

- Create a class Node which has two attributes: data and next. Next is apointer to the next node.
- Create another class which has two attributes: head and tail.

a. addNode() will add a new node to the list:

1. Create a new node.
2. It first checks, whether the head is equal to null which means thelist is empty.
3. If the **list is empty,** both **head and tail** will point to the **newly addednode.**
4. If the list is not empty, the new node will be **added to end** of the listsuch that **tail's next** will point to the **newly added node**. This new node will become the new tail of the list.

b. display() will display the nodes present in the list:

4. Define a node current which **initially points to the head** of the list.
5. Traverse through the list till **current points to null.**

6. Display each node by making current to point to node next to it in each iteration.

a. minNode() will display minimum value node:

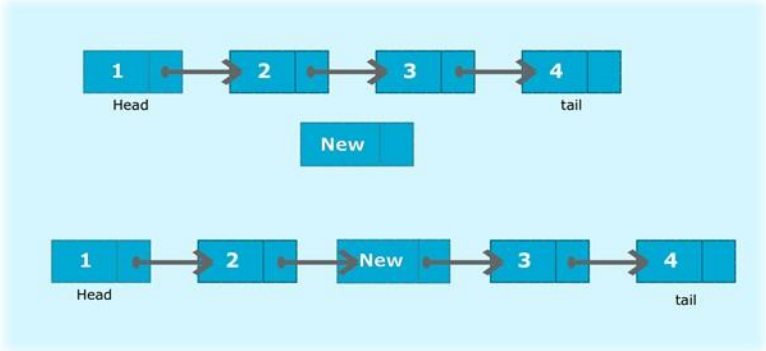
- Define a variable min and initialize it with head's data.
- Node current will point to head.
- Iterate through the list by comparing each node's data with min.
- If min is greater than current's data then, min will hold current's data.
- At the end of the list, variable min will hold minimum value node.
- Display the min value.

a. maxNode() will display maximum value node:

- Define a variable max and initialize it with head's data.
- Node current will point to head.
- Iterate through the list by comparing each node's data with max.
- If max is less than current's data then, max will hold current's data.
- At the end of the list, variable max will hold maximum value node.
- Display the max value.

Java Program to insert a new node at the middle of the singly linked list

In this program, we will create a singly linked list and add a new node at the middle of the list. To accomplish this task, we will calculate the size of the list and divide it by2 to get the mid-point of the list where the new node needs to be inserted.



Consider the above diagram; node 1 represents the head of the original list. Let node New is the new node which needs to be added at the middle of the list. First, we calculate size which in this case is 4. So, to get the mid-point, we divide it by 2 and store it in a variable count. Node current will point to head. First, we iterate through the list till current points to the mid position. Define another node temp which point to node next to current. Insert the New node between current and temp

Algorithm

- Create a class Node which has two attributes: data and next. Next is apointer to the next node.
- Create another class which has two attributes: head and tail.

a. addNode() will add a new node to the list:

5. Create a new node.
6. It first checks, whether the head is equal to null which means thelist is empty.
7. If the list is empty, both head and tail will point to the newly addednode.
8. If the list is not empty, the new node will be added to end of the listsuch that tail's next will point to the newly added node. This new node will become the new tail of the list.

b. display() will display the nodes present in the list:

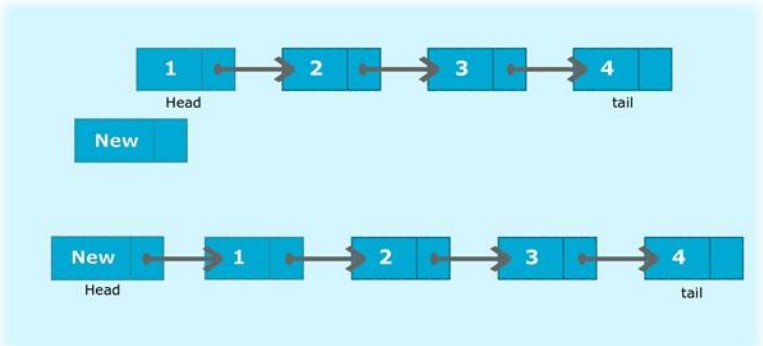
7. Define a node current which initially points to the head of the list.
8. Traverse through the list till current points to null.
9. Display each node by making current to point to node next to it in each iteration.

c. addInMid() will add a new node at the middle of the list:

- It first checks, whether the head is equal to null which means the list is empty.
- If the list is empty, both head and tail will point to a newly added node.
- If the list is not empty, then calculate the size of the list and divide it by 2 to get mid-point of the list.
- Define node current that will iterate through the list until current will point tothe mid node.
- Define another node temp which will point to node next to current.
- The new node will be inserted after current and before temp such that currentwill point to the new node and the new node will point to temp.

Java program to insert a new node at the beginningof the singly linked list

In this program, we will create a singly linked list and add a new node at the beginning of the list. To accomplish this task, we will store head to a temporary node



temp. Make newly added node as the new head of the list. Then, add temp (old head) after new head.

Consider the above list; node 1 represents the head of the original list. Let node New be the new node which needs to be added at the beginning of the list. Node temp will point to head, i.e., 1. Make New as the new head of the list and add temp after new head such that node next to New will be 1.

Algorithm

- Create a class Node which has two attributes: data and next. Next is apointer to the next node.
- Create another class which has two attributes: head and tail.

a. addNode() will add a new node to the list:

1. Create a new node.
2. It first checks, whether the head is equal to null which means thelist is empty.
3. If the list is empty, both head and tail will point to the newly addednode.
4. If the list is not empty, the new node will be added to end of the listsuch that tail's next will point to the newly added node. This new node will become the new tail of the list.

b. display() will display the nodes present in the list:

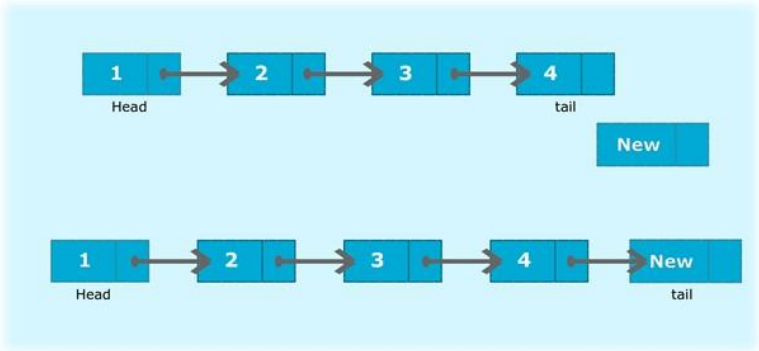
1. Define a node current which initially points to the head of the list.
2. Traverse through the list till current points to null.
3. Display each node by making current to point to node next to it in eachiteration.

C.addAtStart() will add a new node to the beginning of the list:

- It first checks, whether the head is equal to null which means the list isempty.
- If the list is empty, both head and tail will point to a newly added node.
- If the list is not empty then, create temporary node temp will point to head.
- Add the new node as head of the list.
- Temp which was pointing to the old head will be added after the new head.

Java program to insert a new node at the end of the singly linked list

In this program, we will create a singly linked list and add a new node at the end of the list. To accomplish this task, add a new node after the tail of the list such that tail's next will point to the newly added node. Then, make this new node as the new tail of the list.



Consider the above list; node 4 represents the tail of the original list. Let node New isthe new node which needs to be added at the end of the list. Make 4's next to point to New. Make New as the new tail of the list.

Algorithm

- Create a class Node which has two attributes: data and next. Next is apointer to the next node.
- Create another class which has two attributes: head and tail.

a. addNode() will add a new node to the list:

1. Create a new node.
2. It first checks, whether the head is equal to null which means thelist is empty.
3. If the list is empty, both head and tail will point to the newly addednode.
4. If the list is not empty, the new node will be added to end of the listsuch that tail's next will point to the newly added node. This new node will become the new tail of the list.

b. display() will display the nodes present in the list:

1. Define a node current which initially points to the head of the list.
2. Traverse through the list till current points to null.
3. Display each node by making current to point to node next to it in eachiteration.

C. addAtEnd() will add a new node at the end of the list:

- Create a new node.
- It first checks, whether the head is equal to null which means the list isempty.
- If the list is empty, both head and tail will point to a newly added node.
- If the list is not empty, the new node will be added to end of the list such that tail's next will point to a newly added node. This new node will become the new tail of the list.

Java Program to search an element in a singly linkedlist

In this program, we need to search a node in the given singly linked list.

Identity Matrix

To solve this problem, we will traverse through the list using a node current. Current points to head and start comparing searched node data with current node data. If they are equal, set the flag to true and print the message along with the position of the searched node.

For e.g., In the above list, a search node says 4 which can be found at the position 4.

Algorithm

- Create a class Node which has two attributes: data and next. Next is apointer to the next node.
- Create another class which has two attributes: head and tail.

a. addNode() will add a new node to the list:

1. Create a new node.
2. It first checks, whether the head is equal to null which means thelist is empty.
3. If the list is empty, both head and tail will point to the newly addednode.
4. If the list is not empty, the new node will be added to end of the listsuch that tail's next will point to the newly added node. This new node will become the new tail of the list.

b. display() will display the nodes present in the list:

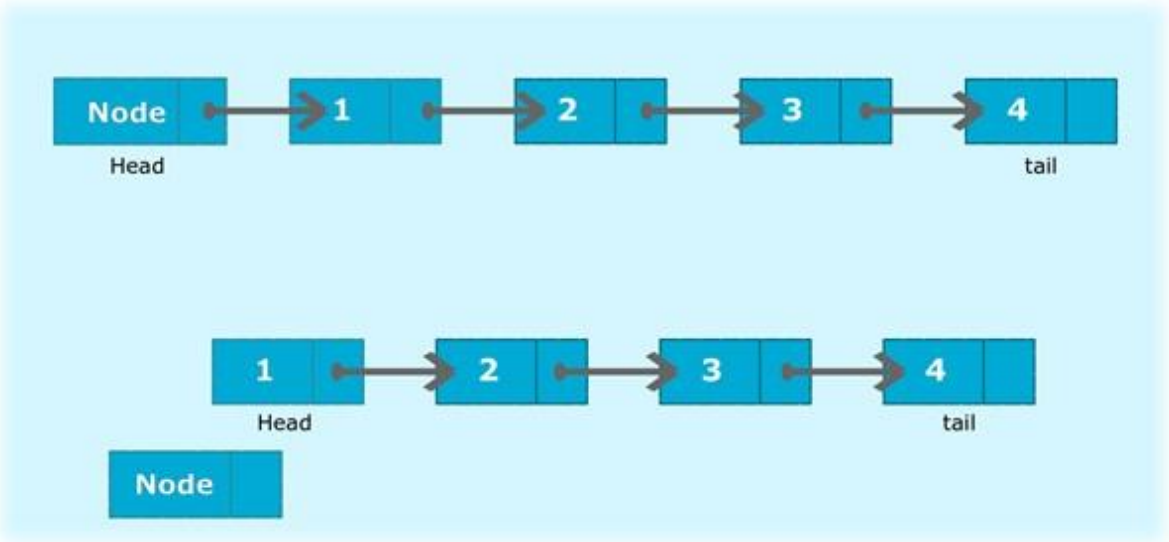
1. Define a node current which initially points to the head of the list.
2. Traverse through the list till current points to null.
3. Display each node by making current to point to node next to it in each iteration.

C. searchNode() will search for a node in the list:

- Variable i will keep track of the position of the searched node.
- The variable flag will store boolean value false.
- Node current will point to head node.
- Iterate through the loop by incrementing current to current.next and i to i +
- Compare each node's data with the searched node if a match is found, set aflag to true.
- If the flag is true, display the position of the searched node.
- Else, display the message "Element is not present in the list".

Java program to delete a node from the beginning ofthe singly linked list

In this program, we will create a singly linked list and delete a node from the beginning of the list. To accomplish this task, we need to make the head pointer pointing to the immediate next of the initial node which will now become the new head node of the list.



Consider the above example; Node was the head of the list. Make head to point to next node in the list. Now, node 1 will become the new head of the list. Thus, deleting the Node.

Algorithm

- Create a class Node which has two attributes: data and next. Next is apointer to the next node.
- Create another class which has two attributes: head and tail.

a. addNode() will add a new node to the list:

1. Create a new node.
2. It first checks, whether the head is equal to null which means thelist is empty.
3. If the list is empty, both head and tail will point to the newly addednode.
4. If the list is not empty, the new node will be added to end of the listsuch that tail's next will point to the newly added node. This new node will become the new tail of the list.

b. display() will display the nodes present in the list:

1. Define a node current which initially points to the head of the list.
2. Traverse through the list till current points to null.
3. Display each node by making current to point to node next to it in each iteration.

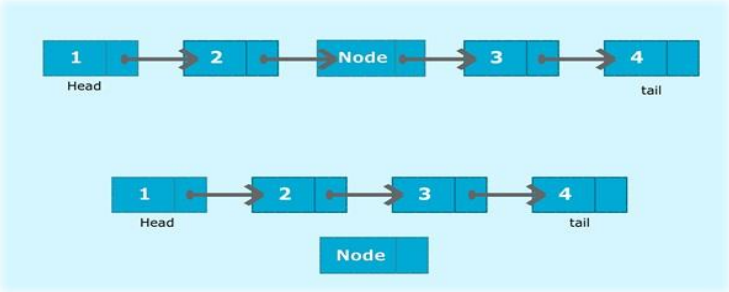
c. deleteFromStart() will delete a node from the beginning of the list:

1. It first checks whether the head is null (empty list) then, display the message "List is empty" and return.
2. If the list is not empty, it will check whether the list has only one node.

3. If the list has only one node, it will set both head and tail to null.
4. If the list has more than one node then, the head will point to the next node inthe list and delete the old head node.

Java program to delete a node from the middle of thesingly linked list

In this program, we will create a singly linked list and delete a node from the middle of the list. To accomplish this task, we will calculate the size of the list and then divide it by 2 to get the mid-point of the list. Node temp will point to head node. We will iterate through the list till midpoint is reached. Now, the temp will point to middle node and node current will point to node previous to temp. We delete the middle node such that current's next node will point to temp's next node.



Consider the above example, mid-point of the above list is 2. Iterate temp from head to mid-point. Now, temp is pointing to the mid node which needs to be deleted. In this case, Node is the middle node which needs to be deleted. The node can be deleted by making node 2's next (current) to point to node 3 (temp's next node). Set temp to null.

Algorithm

- Create a class Node which has two attributes: data and next. Next is apointer to the next node.
- Create another class which has two attributes: head and tail.

a. addNode() will add a new node to the list:

1. Create a new node.
2. It first checks, whether the head is equal to null which means thelist is empty.
3. If the list is empty, both head and tail will point to the newly addednode.
4. If the list is not empty, the new node will be added to end of the listsuch that tail's next will point to the newly added node. This new node will become the new tail of the list.

b. display() will display the nodes present in the list:

1. Define a node current which initially points to the head of the list.

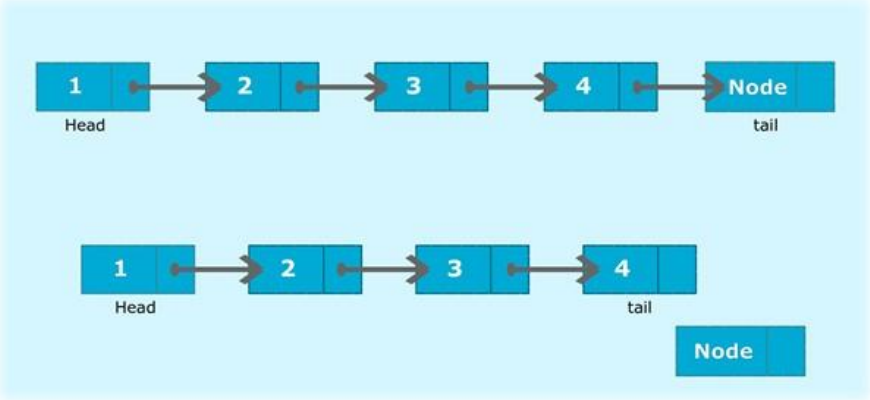
2. Traverse through the list till current points to null.
3. Display each node by making current to point to node next to it in each iteration.

c. deleteFromMid() will delete a node from the middle of the list:

1. It first checks whether the head is null (empty list) then, display the message "List is empty" and return.
2. If the list is not empty, it will check whether the list has only one node.
3. If the list has only one node, it will set both head and tail to null.
4. If the list has more than one node then, calculate the size of the list and divide it by 2 to get the mid-point of the list.
5. Declare a node temp which will point to head and node current will point to node previous to temp.
6. Traverse through the list till temp points to a middle node. If current not point to null then, delete the middle node(temp) by making current's next to point to temp's next. Else, both head and tail will point to node next to temp and delete the middle node by setting the temp to null.

Java program to delete a node from the end of thesingly linked list

In this program, we will create a singly linked list and delete a node from the end of the list. To accomplish this task, we first find out the second last node of the list.Then, make second last node as the new tail of the list. Then, delete the last node of the list.



In the above example, Node was the tail of the list. Traverse through the list to find out the second last node which in this case is node 4. Make node 4 as the tail of the list. Node 4's next will point to null.

Algorithm

- Create a class Node which has two attributes: data and next. Next is apointer to the next node.

- Create another class which has two attributes: head and tail.

a. addNode() will add a new node to the list:

1. Create a new node.
2. It first checks, whether the head is equal to null which means thelist is empty.
3. If the list is empty, both head and tail will point to the newly addednode.
4. If the list is not empty, the new node will be added to end of the listsuch that tail's next will point to the newly added node. This new node will become the new tail of the list.

b. display() will display the nodes present in the list:

1. Define a node current which initially points to the head of the list.
2. Traverse through the list till current points to null.
3. Display each node by making current to point to node next to it in each iteration.

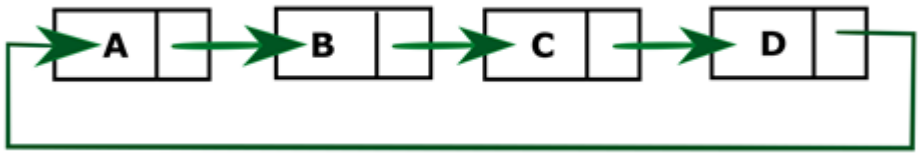
a. DeleteFromEnd() will delete a node from the end of the list:

1. It first checks whether the head is null (empty list) then, display the message "List isempty" and return.
2. If the list is not empty, it will check whether the list has only one node.
3. If the list has only one node, it will set both head and tail to null.
4. If the list has more than one node then, traverse through the list till node currentpoints to second last node in the list.
5. Node current will become the new tail of the list.
6. Node next to current will be made null to delete the last node.

Circular Linked List:

The circular linked list is a kind of linked list. First thing first, the node is an element of the list, and it has two parts that are, data and next. Data represents the data stored in the node and next is the pointer that will point to next node. Head will point to the first element of the list, and tail will point to the last element in the list. In the simple linked list, all the nodes will point to their next element and tail will point to null.

The circular linked list is the collection of nodes in which tail node also point back to head node. The diagram shown below depicts a circular linked list. Node A represents head and node D represents tail. So, in this list, A is pointing to B, B is pointing to C and C is pointing to D but what makes it circular is that node D is pointing back to node A.



Java program to create and display a Circular Linked List

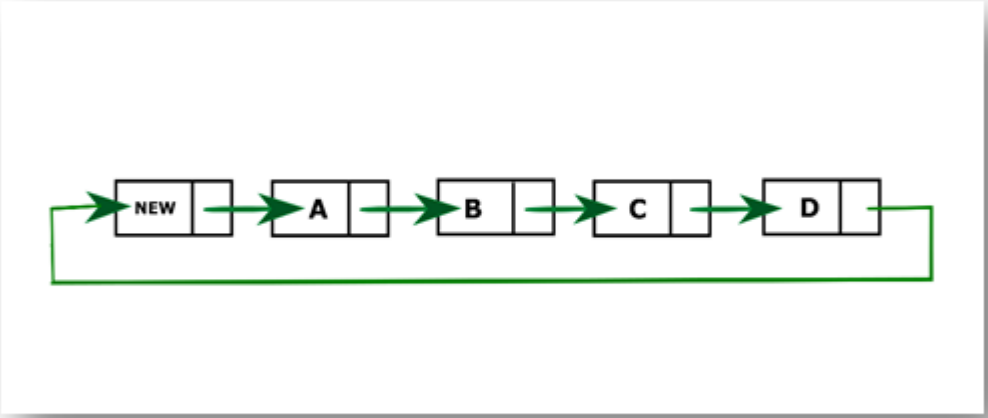
- Define a Node class which represents a node in the list. It has two properties data and next which will point to the next node.
- Define another class for creating the circular linked list and it has two nodes: head and tail. It has two methods: add() and display() .
- add() will add the node to the list:
    - It first checks whether the head is null, then it will insert the node as the head.
    - Both head and tail will point to the newly added node.
    - If the head is not null, the new node will be the new tail, and the new tail will point to the head as it is a circular linked list.

a. display() will show all the nodes present in the list.

- Define a new node 'current' that will point to the head.
- Print current.data till current will points to head
- Current will point to the next node in the list in each iteration.

Java program to insert a new node at the beginning of the Circular Linked List

In this program, we will create a circular linked list and insert every new node at the beginning of the list. If the list is empty, then head and tail will point to the newly added node. If the list is not empty, then we will store the data of the head into a temporary node temp and make new node as the head. This new head will point to the temporary node. In simple words, the newly added node will be the first node(head) and previous head(temp) will become the second node of the list.

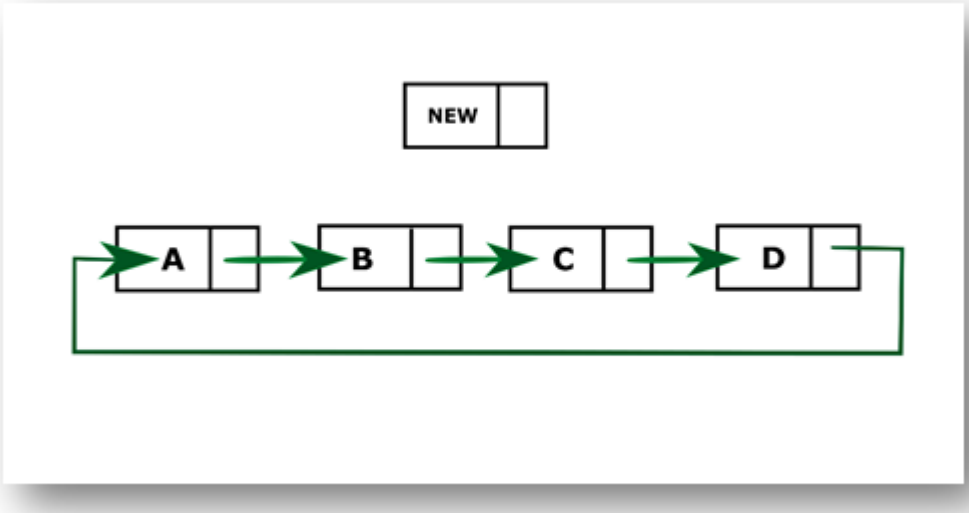


- addAtStart() will add the node to the beginning of the list:
  - It first checks whether the head is null (empty list), then it will insert the node as the head.
  - Both head and tail will point to newly added node.
  - If the list is not empty, then the newly added node will become the new head, and it will point to previous head.

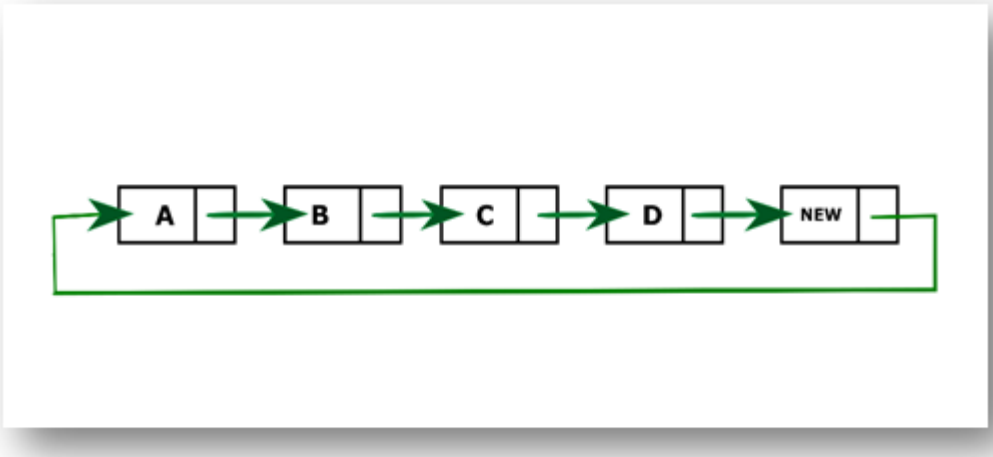
Java program to insert a new node at the end of the Circular Linked List

In this program, we will create a circular linked list and insert every new node at the end of the list. If the list is empty, then head and tail will point to the newly added node. If the list is not empty, the newly added node will become the new tail of the list. The previous tail will point to new node as its next node. Since it is a circular linked list; the new tail will point to head. In other words, the new node will become last node (tail) of the list, and the previous tail will be the second last node.

Identity Matrix



After inserting new node to the end of the list



New represents the newly added node. D is the previous tail. When new is added to the end of the list, it will become new tail and D will point to new.

Algorithm

- Define a Node class which represents a node in the list. It has two properties data and next which will point to the next node.
- Define another class for creating the circular linked list and it has two nodes: head and tail. It has two methods: addAtEnd() and display() .

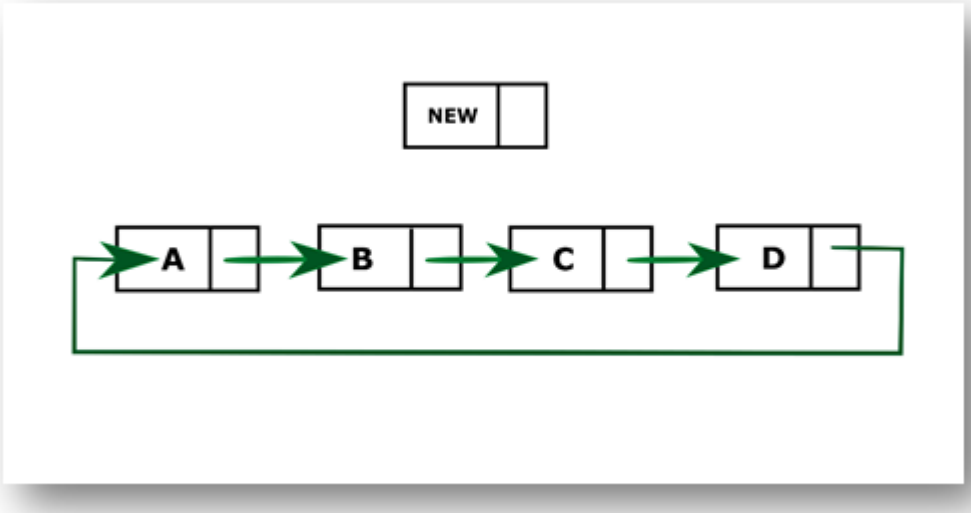
- addAtEnd() will add the node to the end of the list:
  - It first checks whether the head is null (empty list), then it will insert the node as the head.
  - Both head and tail will point to the newly added node.
  - If the list is not empty, then the newly added node will become the new tail, and previous tail will point to new node as its next node. The new tail will point to the head.

a. display() will show all the nodes present in the list.

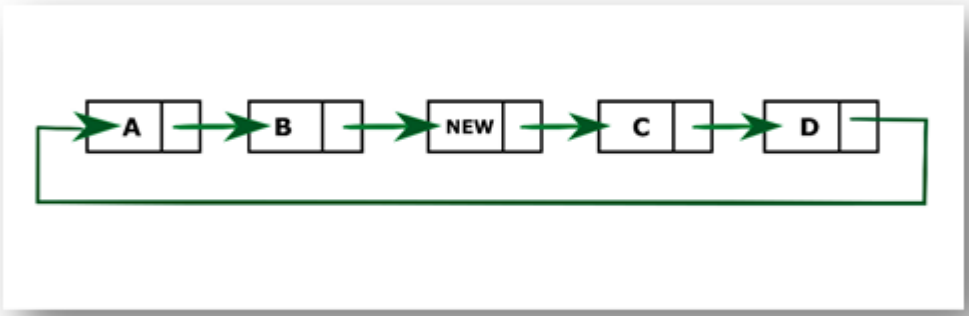
- Define a new node 'current' that will point to the head.
- Print current.data till current will points to head again.
- Current will point to the next node in the list in each iteration.

Java program to insert a new node at the middle of the Circular Linked List

In this program, we create a circular linked list and insert a new node in the middle of the list. If the list is empty, both head and tail will point to new node. If the list is not empty, then we will calculate the size of the list and divide it by 2 to get the mid-point of the list where the new node needs to be inserted.



After inserting the new node in the middle of the list.



Consider the above diagram; the new node needs to be added to the middle of the list. First, we calculate the size which in this case is 4. So, to get the mid-point, we divide it by 2 and store it in a variable count. We will define two nodes current, and temp such that temp will point to head, and current will point to the node previous to temp. We iterate through the list till mid-point is reached by incrementing temp to temp.next then, insert the new node in between current and temp. Current'next node will be new and the new's next node will be temp.

Algorithm

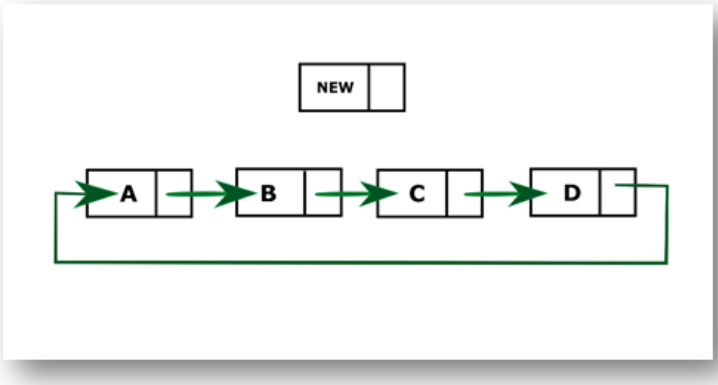
- Define a Node class which represents a node in the list. It has two properties data and next which will point to the next node.
- Define another class for creating the circular linked list and it has two nodes: head and tail. Variable size stores the size of the list. It has two methods: addInMid() and display() .
- addInMid() will add the node to the middle of the list:
    - It first checks whether the head is null (empty list), then it will insert the node as the head.
    - Both head and tail will point to the newly added node.
    - If the list is not empty, then we calculate size and divide it by 2 to get the mid-point.
    - Define node temp that will point to head and current will point to a node previous to temp.
    - Iterate through the list until the middle of the list is reached by incrementing temp to temp.next.
    - The new node will be inserted after current and before temp such that current will point to the new node and the new node will point to temp.

a. display() will show all the nodes present in the list.

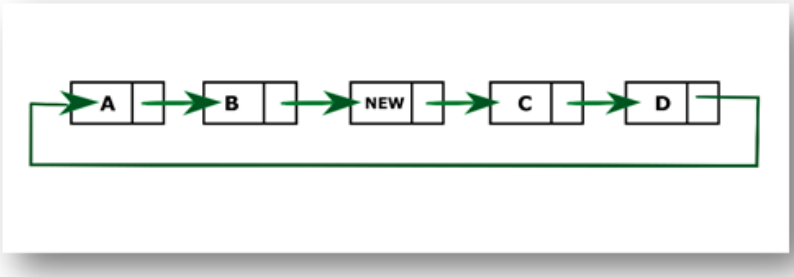
- Define a new node 'current' that will point to the head.
- Print current.data till current will points to head again.
- Current will point to the next node in the list in each iteration.

Java program to insert a new node at the middle of the Circular Linked List

In this program, we create a circular linked list and insert a new node in the middle of the list. If the list is empty, both head and tail will point to new node. If the list is not empty, then we will calculate the size of the list and divide it by 2 to get the mid-point of the list where the new node needs to be inserted.



After inserting the new node in the middle of the list.



Consider the above diagram; the new node needs to be added to the middle of the list. First, we calculate the size which in this case is 4. So, to get the mid-point, we divide it by 2 and store it in a variable count. We will define two nodes current, and temp such that temp will point to head, and current will point to the node previous to temp. We iterate through the list till mid-point is reached by incrementing temp

to temp.next then, insert the new node in between current and temp. Current'next node will be new and the new's next node will be temp.

Algorithm

- Define a Node class which represents a node in the list. It has two properties data and next which will point to the next node.
- Define another class for creating the circular linked list and it has two nodes: head and tail. Variable size stores the size of the list. It has two methods: addInMid() and display() .
- addInMid() will add the node to the middle of the list:
    - It first checks whether the head is null (empty list), then it will insert the node as the head.
    - Both head and tail will point to the newly added node.
    - If the list is not empty, then we calculate size and divide it by 2 to get the mid-point.
    - Define node temp that will point to head and current will point to a node previous to temp.
    - Iterate through the list until the middle of the list is reached by incrementing temp to temp.next.
    - The new node will be inserted after current and before temp such that current will point to the new node and the new node will point to temp.

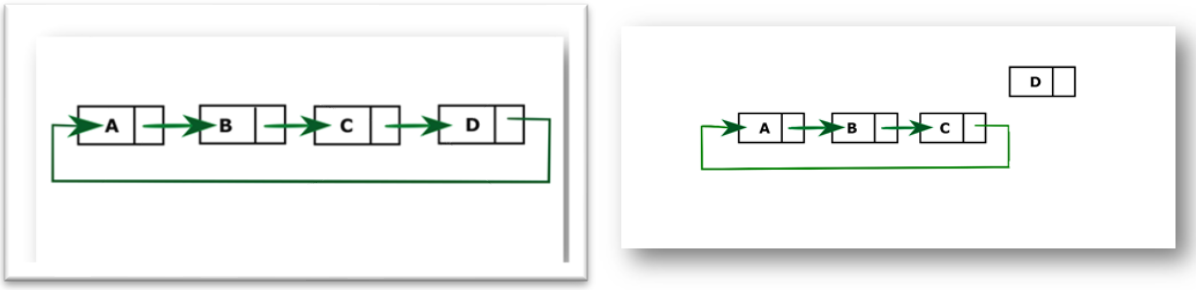
a. display() will show all the nodes present in the list.

- Define a new node 'current' that will point to the head.
- Print current.data till current will points to head again.
- Current will point to the next node in the list in each iteration.

Java program to delete a node from the end of the Circular Linked List

In this program, we will create a circular linked list and delete a node from the end of the list. If the list is empty, it will display the message "List is empty". If the list is not empty, we will loop through the list till second last node is reached. We will make second last node as the new tail, and this new tail will point to head and delete the previous tail.

Circular linked list after deleting node from end



Here, in the above list, D is the last node which needs to be deleted. We will iterate through the list till C. Make C as new tail and C will point back to head A.

Algorithm

- Define a Node class which represents a node in the list. It has two properties data and next which will point to the next node.
- Define another class for creating the circular linked list and it has two nodes: head and tail. It has two methods: deleteEnd() and display() .
- deleteEnd() will delete the node from the end of the list:
    - It first checks whether the head is null (empty list) then, it will return from the function as there is no node present in the list.
    - If the list is not empty, it will check whether list has only one node.
    - If the list has only one node, it will set both head and tail to null.
    - If the list has more than one node then, iterate through the loop till current.next!= tail.
    - Now, current will point to the node previous to tail. Make current as new tail and tail will point to head thus, deletes the node from last.

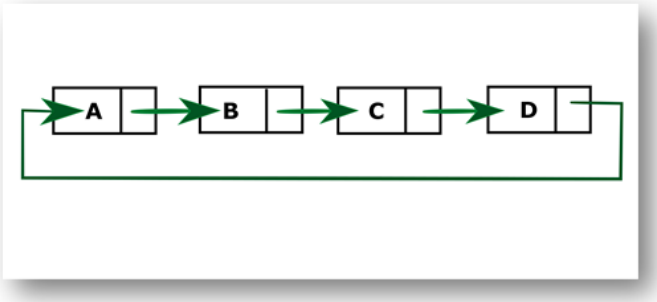
a. display() will show all the nodes present in the list.

- Define a new node 'current' that will point to the head.
- Print current.data till current will points to head again.
- Current will point to the next node in the list in each iteration.

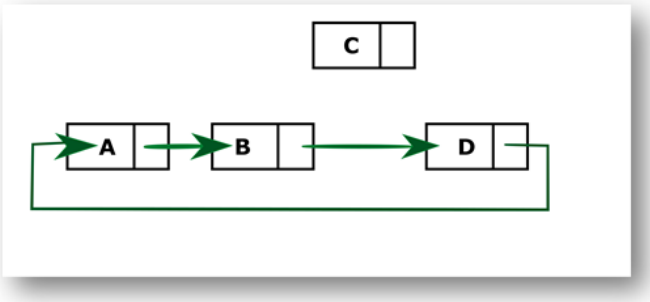
Java program to delete a node from the middle of the Circular Linked List

In this program, we will create a circular linked list and delete a node from the middle of the list. If the list is empty, display the message "List is empty". If the list is not empty, we will calculate the size of the list and then divide it by 2 to get the mid-point of the list. We maintain two pointers temp and current. Current will point to the previous node of temp. We will iterate through the list until mid-point is reached then the current will point to the middle node. We delete middle node such that current's next will be temp's next node.

Identity Matrix



Circular linked list after deleting node from middle of the list



Consider the above list. Size of the list is 4. Mid-point of the node is 2. To remove C from the list, we will iterate through the list till mid-point. Now current will point to B and temp will point to C. C will be removed when B will point to D.

Algorithm

- Define a Node class which represents a node in the list. It has two properties data and next which will point to the next node.

- Define another class for creating the circular linked list and it has two nodes: head and tail. It has a variable size and two methods: deleteMid() and display() .
- deleteMid() will delete the node from the middle of the list:
    - It first checks whethe the head is null (empty list) then, it will return from the function as there is no node present in the list.
    - If the list is not empty, it will check whether the list has only one node.
    - If the list has only one node, it will set both head and tail to null.
    - If the list has more than one node then, it will calculate the size of the list. Divide the size by 2 and store it in the variable count.
    - Temp will point to head, and current will point to the previous node to temp.
    - Iterate the list till current will point middle node of the list.
    - Current will point to node next to temp, i.e., removes the node next to current.

a. display() will show all the nodes present in the list.

- Define a new node 'current' that will point to the head.
- Print current.data till current will points to head again.
- Current will point to the next node in the list in each iteration.

Applet Introduction

Java is general purpose, object oriented programming language. We can develop two types of application.
1) Stand Alone Application
2) Web Applets

- "**applets"** are small applications that are accessed on an Internet server, transported over the **Internet, automatically installed, and run as part** of a Web document.
- After an applet arrives on the client, it has limited access to resources, so that it can produce an arbitrary **multimedia user** interface and run **complex computations** without introducing the risk of viruses or breaching data integrity.

Example:

```
public class SimpleApplet extends Applet
{
        public void paint(Graphics g)
        {
                g.drawString("A Simple Applet", 20, 20);
        }
}
```

This applet begins with two import statements.

- The first imports the Abstract Window Toolkit (AWT) classes. Applets interact with the user through the AWT(Abstract Window Toolkit) applet that you create must be a subclass (either directly or indirectly) of Applet class. import the Graphics class from AWT package.
- The second import statement imports the applet package, which contains the class Applet. Every applet that you create must be a subclass of Applet.
- The next line in the program declares the class SimpleApplet. This class must be declared as public, because it will be accessed by code that is outside the program.
- paint( ) is called each time that the applet must redisplay its output. This situation can occur for several reasons.
  For example, the window in which the applet is running can be overwritten by another window and then uncovered. Or, the applet window can be minimized and then restored. paint( ) is also called when the applet begins execution. Whatever the cause, whenever the applet must redraw its output, paint( ) is called.
- The paint( ) method has one parameter of type Graphics. This parameter contains the graphics context, which describes the graphics environment in which the applet is running. This context is used whenever output to the applet is required.
- Inside paint( ) is a call to drawString( ), which is a member of the Graphics class. This method outputs a string beginning at the specified X,Y location. It has the following general form:

  ```
  void drawString(String message, int x, int y)
  ```

In a Java window, the upper-left corner is location 0,0. The call to drawString( ) in the applet causes the message “A Simple Applet” to be displayed beginning at location 20,20.

Notice that the applet does not have a main( ) method. Unlike Java programs, applets do not begin execution at main( ). In fact, most applets don’t even have a main( )

method. Instead, an applet begins execution when the name of its class is passed to an applet viewer or to a network browser.

In fact, there are two ways in which you can run an applet:

■ Executing the applet within a Java-compatible Web browser.
    ■ Using an applet viewer, such as the standard SDK tool, appletviewer. An applet viewer executes your applet in a window. This is generally the fastest and easiest way to test your applet.

Each of these methods is described next. To execute an applet in a Web browser, you need to write a short HTML text file that contains the appropriate APPLET tag. Here is the HTML file that executesSimpleApplet:

```
<applet code="SimpleApplet" width=200 height=60>
</applet>
```

- The width and height statements specify the dimensions of the display area used by the applet
- After you create this file, you can execute your browser and then load this file, which causes SimpleApplet to be executed.

To execute SimpleApplet with an applet viewer, you may also execute the HTML file shown earlier. For example, if the preceding HTML file is called RunApp.html, then the following command line will run SimpleApplet:

```
C:\>appletviewer RunApp.html
```

However, a more convenient method exists that you can use to speed up testing. Simply include a comment at the head of your Java source code file that contains the APPLET tag. By doing so, your code is documented with a prototype of the necessary HTML statements, and you can test your compiled applet merely by starting the applet viewer with your Java source code file. If you use this method, the SimpleApplet source file looks like this:

```
import java.awt.*;
import java.applet.*;

/*
<applet code="SimpleApplet" width=200 height=60>
```

```
</applet>
*/
public class SimpleApplet extends Applet
{
        public void paint(Graphics g)
        {
                g.drawString("A Simple Applet", 20, 20);
        }
}
```

In general, you can quickly iterate through applet development by using these three steps:

1. Edit a Java source file.
2. Compile your program.
3. Execute the applet viewer, specifying the name of your applet's source file. The applet viewer will encounter the APPLET tag within the comment and execute your applet.

The window produced by SimpleApplet, as displayed by the applet viewer, is shown in the following illustration:

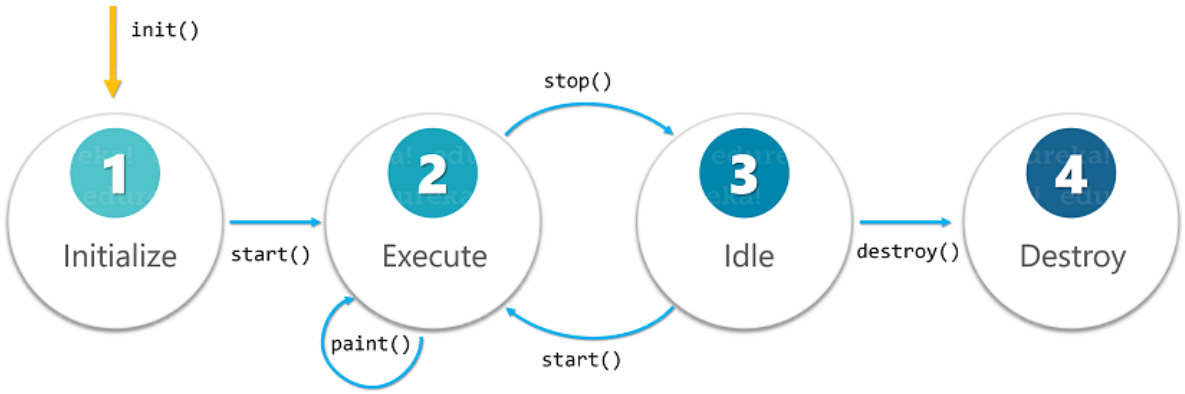
While the subject of applets is more fully discussed later in this book, here are the key points that you should remember now:

- Applets do not need a main( ) method.
- Applets must be run under an applet viewer or a Java-compatible browser.
- User I/O is not accomplished with Java's stream I/O classes. Instead, applets use the interface provided by the AWT.
- All applets are subclasses (either directly or indirectly) of Applet
- Applets are not stand-alone programs. Instead, they run within either a web browser or an programming java.

An Applet Skeleton

All but the most trivial applets override a set of methods that provides the basic mechanism by which the browser or applet viewer interfaces to the applet and controls its execution.

- Four of these methods—init( ), start( ), stop( ), and destroy( )—are defined by Applet.
- Another, paint( ), is defined by the AWT Component class.
- Simple applets will not need to define all of them. These five methods can be assembled into the skeleton




```
// An Applet skeleton.

import java.awt.*;
import java.applet.*;
/*
<applet code="p11" width=300 height=100>
</applet>
*/
public class p11 extends Applet {
        String s;
// Called first.
public void init()
{
// initialization
s="init.....";
}
/* Called second, after init(). Also called whenever the applet is restarted. */
public void start()
{
// start or resume execution
s+="start.....";
}
public void update(Graphics g)
{s+="update.....";
// redisplay your window, here.
}

// Called when the applet is stopped.
public void stop() {
// suspends execution
s+="stop.....";
```

```
}
/* Called when applet is terminated. This is the last
method executed. */
public void destroy() {
// perform shutdown activities
s+="destroy…..";

}
// Called when an applet's window must be restored.
public void paint(Graphics g) {
// redisplay contents of window
s+="paint…..";
g.drawString(s,23,23);
}
}
```

Applet Initialization and Termination

It is important to understand the order in which the various methods shown in the skeleton are called. When an **applet begins**, the AWT calls the following methods, in this sequence:

1. **init( )**
2. **start( )**
3. **paint( )**

When an **applet is terminated**, the following sequence of method calls takes place:

1. **stop( )**
2. **destroy( )**

init( )

The **init( )** method is the first method to be called. This is where you should **initialize variables.** This method is called only once during the run time of your applet.

start( )

The **start( )** method is called after **init( )**. It is also called to **restart an applet** after it has been stopped. Whereas **init( ) is called once—the first time an applet is loaded—start(**

) is called each time an applet's HTML document is displayed onscreen. So, if a user leaves a web page and comes back, the applet resumes execution at start( ).

paint( )

The paint( ) method is called each time your applet's output must be redrawn.

- The window in which the applet is running may be overwritten by another window and then uncovered. Or the applet window may be minimized and then restored.
- paint( ) is also called when the applet begins execution.
- whenever the applet must redraw its output, paint( ) is invoked.
- The paint( ) method has one parameter of type Graphics. This parameter will contain the graphics context, which describes the graphics environment in which the applet is running. This context is used whenever output to the applet is required.

stop( )

The stop( ) method is called when a web browser leaves the HTML document containing the applet—when it goes to another page,
for example. When stop( ) is invoked, the applet is probably running. You should use stop( ) to suspend threads that don't need to run when the applet is not visible. You can restart them when start( ) is called if the user returns to the page.

destroy( )

The destroy( ) method is called when the environment determines that your applet needs to be removed completely from memory. At this point, you should free up any resources the applet may be using. The stop( ) method is always called before destroy( ).

Overriding update( )

In some situations, your applet may need to override another method defined by the AWT, called update( ). This method is called when your applet has requested that a portion of its window be redrawn. The default version of update( ) first fills an applet with the default background color and then calls paint( ). If you fill the background using a different color in paint( ), the user will experience a flash of the default background each time update( ) is called—that is, whenever the window is repainted.

One way to avoid this problem is to override the **update( )** method so that it performs all necessary display activities. Then have **paint( )** simply call **update( )**. Thus, for some applications, the applet skeleton will override **paint( )** and **update( )**, as shown here:

```
public void update(Graphics g) {
// redisplay your window, here.
}
public void paint(Graphics g) {
update(g);
}
```

Simple Applet Display Methods

Applets are displayed in a window and they use the AWT to perform input and output. Although we will examine the methods, procedures, and techniques necessary to fully handle the AWT windowed environment in subsequent chapters, a few are described here, because we will use them to write sample applets.
To output a string to an applet, use **drawString( )**, which is a member of the **Graphics** class. Typically, it is called from within either **update( )** or **paint( )**. It has the following general form:

void drawString(String *message*, int *x*, int *y*)

Here, *message* is the string to be output beginning at *x,y.* In a Java window, the upper-left corner is location 0,0. The **drawString( )** method will not recognize newline characters. If you want to start a line of text on another line, you must do so manually, specifying the precise X,Y location where you want the line to begin.

- To set the background color of an applet's window, use **setBackground( )**.
- To set the foreground color (the color in which text is shown, for example), use **setForeground( )**. These methods are defined by **Component**, and they have the following general forms:

void setBackground(Color *newColor*)
void setForeground(Color *newColor*)

Here, *newColor* specifies the new color. The class **Color** defines the constants shown here that can be used to specify colors:

Color.black	Color.magenta
Color.blue	Color.orange
Color.cyan	Color.pink
Color.darkGray	Color.red
Color.gray	Color.white
Color.green	Color.yellow
Color.lightGray	

For example, this sets the background color to green and the text color to red:

```
setBackground(Color.green);
setForeground(Color.red);
```

A good place to set the foreground and background colors is in the init( ) method. Of course, you can change these colors as often as necessary during the execution of your applet. The default foreground color is black. The default background color is light gray.

You can obtain the current settings for the background and foreground colors by calling getBackground( ) and getForeground( ), respectively. They are also defined by Component and are shown here:

```
Color getBackground( )
Color getForeground( )
```

/* A simple applet that sets the foreground and background colors and outputs a string. */

```
import java.awt.*;
import java.applet.*;

/*
        <applet code="Sample" width=300 height=50>
        </applet>
*/

public class Sample extends Applet{
String msg;
// set the foreground and background colors.
public void init() {
        setBackground(Color.cyan);
        setForeground(Color.red);
        msg = "Inside init( ) --";
}
// Initialize the string to be displayed.
public void start() {
        msg += " Inside start( ) --";
}
// Display msg in applet window.
public void paint(Graphics g) {
        msg += " Inside paint( ).";
        g.drawString(msg, 10, 30);
        }
```

}

The **repaint( )** method is defined by the AWT. It causes the AWT run-time system to execute a call to your applet's **update( )** method, which, in its default implementation, calls **paint( )**. Thus, for another part of your applet to output to its window, simply store the output and then call **repaint( )**. The AWT will then execute a call to **paint( )**, which can display the stored information. For example, if part of your applet needs to output a string, it can store this string in a **String** variable and then call **repaint( )**. Inside **paint( )**, you will output the string using **drawString( )**. The **repaint( )** method has four forms. Let's look at each one, in turn. The simplest version of **repaint( )** is shown here:

void repaint( )

This version causes the entire window to be repainted. The following version specifies a region that will be repainted:

void repaint(int *left*, int *top*, int *width*, int *height*)

the coordinates of the upper-left corner of the region are specified by *left* and *top*, and the width and height of the region are passed in *width* and *height.* These dimensions are specified in pixels. You save time by specifying a region to repaint. Window updates are costly in terms of time. If you need to update only a small portion of the window, it is more efficient to repaint only that region.

Calling **repaint( )** is essentially a request that your **applet** be repainted sometime soon. However, if your system is slow or busy, **update( )** might not be called immediately. Multiple requests for repainting that occur within a short time can be collapsed by the AWT in a manner such that **update( )** is only called sporadically. This can be a problem in many situations, including animation, in which a consistent update time is necessary. One solution to this problem is to use the following forms of **repaint( )**:

void repaint(long *maxDelay*)
void repaint(long *maxDelay*, int *x*, int *y*, int *width*, int *height*)

Here, *maxDelay* specifies the maximum number of milliseconds that can elapse before **update()** is called. Beware, though. If the time elapses before **update( )** can be called, it isn't called. There's no return value or exception thrown, so you must be careful.

A Simple Banner Applet

```
/* A simple banner applet. This applet creates a thread that scrolls the message contained in msg right to left across the applet's window. */
import java.awt.*;
import java.applet.*;

/*
<applet code="SimpleBanner" width=300 height=50> </applet>
*/

public class SimpleBanner extends Applet implements Runnable {
String msg = " A Simple Moving Banner.";
Thread t = null;
int state;
boolean stopFlag;
// Set colors and initialize thread.
public void init() {
setBackground(Color.cyan);
setForeground(Color.red);
}
// Start thread
public void start() {
t = new Thread(this);
stopFlag = false;
t.start();
}
// Entry point for the thread that runs the banner.
public void run() {
char ch;
// Display banner
for( ; ; ) {
try {
repaint();
Thread.sleep(250);
ch = msg.charAt(0);
msg = msg.substring(1, msg.length());
msg += ch;
if(stopFlag)
break;
} catch(InterruptedException e) {}
```

```
}
}
// Pause the banner.
public void stop() {
stopFlag = true;
t = null;
}
// Display the banner.
public void paint(Graphics g) {
g.drawString(msg, 50, 30);
}
}
```

Using the Status Window

In addition to displaying information in its window, an applet can also output a message to the status window of the browser or applet viewer on which it is running. To do so, call **showStatus( )** with the string that you want displayed. The status window is a good place to give the user feedback about what is occurring in the applet, suggest options, or possibly report some types of errors. The status window also makes an excellent debugging aid, because it gives you an easy way to output information about your applet.

```
// Using the Status Window.
import java.awt.*;
import java.applet.*;
/*
<applet code="StatusWindow" width=300 height=50>
</applet>
*/
public class StatusWindow extends Applet{
public void init() {
setBackground(Color.cyan);
}
// Display msg in applet window.
public void paint(Graphics g) {
g.drawString("This is in the applet window.", 10, 20);
showStatus("This is shown in the status window.");
}
}
```

getDocumentBase( ) and getCodeBase( )

Often, you will create applets that will need to explicitly load media and text. Java will allow the applet to load data from the **directory** holding the HTML **file that started the applet** (the *document base*) and the directory from which the applet's **class file was loaded** (the *code base*). These directories are returned as **URL** objectsby **getDocumentBase( )** and **getCodeBase( )**. They can be concatenated with a string that names the file you want to load.

```
import java.awt.*;
import java.applet.*;
import java.net.*;
/*
<applet code="Bases" width=300 height=50>
</applet>
*/

public class Bases extends Applet
{
// Display code and document bases.
        public void paint(Graphics g)
        {
                String msg;
                URL url = getCodeBase(); // get code base
                msg = "Code base: " + url.toString();
                g.drawString(msg, 10, 20);
                url = getDocumentBase(); // get document base
                msg = "Document base: " + url.toString();
                g.drawString(msg, 10, 40);
        }
}
```

Graphics

Drawing Lines

Lines are drawn by means of the **drawLine( )** method, shown here:

void drawLine(int *startX*, int *startY*, int *endX*, int *endY*)

drawLine( ) displays a line in the current drawing color that begins at *startX*,*startY* and ends
at *endX*,*endY*.

Drawing Rectangles

The drawRect( ) and fillRect( ) methods display an outlined and filled rectangle, respectively.
They are shown here:

void drawRect(int *top*, int *left*, int *width*, int *height*)
void fillRect(int *top*, int *left*, int *width*, int *height*)

The upper-left corner of the rectangle is at *top*,*left.* The dimensions of the rectangle are specified by *width* and *height.*

To draw a rounded rectangle, use drawRoundRect( ) or fillRoundRect( ), both shown here:

void drawRoundRect(int *top*, int *left*, int *width*, int *height*, int *xDiam*, int *yDiam*)
void fillRoundRect(int *top*, int *left*, int *width*, int *height*, int *xDiam*, int *yDiam*)
Note xDiam arcwidth yDiam archight

Drawing Ellipses and Circles

To draw an ellipse, use drawOval( ). To fill an ellipse, use fillOval( ). These methods are shown here:

void drawOval(int *top*, int *left*, int *width*, int *height*)
void fillOval(int *top*, int *left*, int *width*, int *height*)

The ellipse is drawn within a bounding rectangle whose upper-left corner is specified by *top*,*left* and whose width and height are specified by *width* and *height.* To draw a circle, specify
a square as the bounding rectangle.

Drawing Arcs

Arcs can be drawn with drawArc( ) and fillArc( ), shown here:

void drawArc(int *top*, int *left*, int *width*, int *height*, int *startAngle*, int *sweepAngle*)
void fillArc(int *top*, int *left*, int *width*, int *height*, int *startAngle*, int *sweepAngle*)

The arc is bounded by the rectangle whose upper-left corner is specified by *top*,*left* and whose
width and height are specified by *width* and *height.* The arc is drawn from *startAngle* through
the angular distance specified by *sweepAngle.* Angles are specified in degrees. Zero degrees is
on the horizontal, at the three o'clock position. The arc is drawn counterclockwise if *sweepAngle* is positive, and clockwise if *sweepAngle* is negative. Therefore, to draw an arc from twelve o'clock to six o'clock, the start angle would be 90 and the sweep angle 180.

Drawing Polygons

It is possible to draw arbitrarily shaped figures using **drawPolygon( )** and **fillPolygon( )**,
shown here:

void drawPolygon(int *x*[ ], int *y*[ ], int *numPoints*)
void fillPolygon(int *x*[ ], int *y*[ ], int *numPoints*)

```
int x[] = { 70, 150, 190, 80, 100 };

 int y[] = { 80, 110, 160, 190, 100 };

 g.drawPolygon (x, y, 5);
```

The polygon's endpoints are specified by the coordinate pairs contained within the *x* and *y* arrays. The number of points defined by *x* and *y* is specified by *numPoints.* There are alternative forms of these methods in which the polygon is specified by a **Polygon** object.

Working with Color

Java supports color in a portable, device-independent fashion. The AWT color system allows you to specify any color you want. It then finds the best match for that color, given the limits of the display hardware currently executing your program or applet. Thus, your code does not need to be concerned with the differences in the way color is supported by various hardware
devices. Color is encapsulated by the **Color** class.

Color defines several constants (for example, Color.black) to specify a number of common colors. You can also create your own colors, using one of the color constructors. Three commonly used forms are shown here:

Color(int *red*, int *green*, int *blue*)
Color(float *red*, float *green*, float *blue*)

The first constructor takes three integers that specify the color as a mix of red, green, and blue. These values must be between 0 and 255, as in this example:

new Color(255, 100, 100); // light red

The final constructor, Color(float, float, float), takes three float values (between 0.0 and 1.0)
that specify the relative mix of red, green, and blue.

Once you have created a color, you can use it to set the foreground and/or background color by using the setForeground( ) and setBackground( ) methods. You can also select it as the current drawing color.

Setting the Current Graphics Color

By default, graphics objects are drawn in the current foreground color. You can change this
color by calling the Graphics method setColor( ):

void setColor(Color *newColor*)
Here, *newColor* specifies the new drawing color.

You can obtain the current color by calling getColor( ), shown here:
Color getColor( )

```
// Demonstrate color.
import java.awt.*;
import java.applet.*;
/*
<applet code="ColorDemo" width=300 height=200>
</applet>
*/
public class ColorDemo extends Applet
```

```
{
// draw lines
        public void paint(Graphics g)
        {
                Color c1 = new Color(255, 100, 100);
                Color c2 = new Color(100, 255, 100);
                Color c3 = new Color(100, 100, 255);
                g.setColor(c1);
                g.drawLine(0, 0, 100, 100);
                g.drawLine(0, 100, 100, 0);
                g.setColor(c2);
                g.drawLine(40, 25, 250, 180);
                g.drawLine(75, 90, 400, 400);
                g.setColor(c3);
                g.drawLine(20, 150, 400, 40);
                g.drawLine(5, 290, 80, 19);
                g.setColor(Color.red);
                g.drawOval(10, 10, 50, 50);
                g.fillOval(70, 90, 140, 100);
                g.setColor(Color.blue);
                g.drawOval(190, 10, 90, 30);
                g.drawRect(10, 10, 60, 50);
                g.setColor(Color.cyan);
                g.fillRect(100, 10, 60, 50);
                g.drawRoundRect(190, 10, 60, 50, 15, 15);
        }
}
```

Working with font

To draw text to the screen, first, you need to create an instance of the Font class.

Font objects represent an individual font; that is, its name, style (bold, italic), and point size.

Font names are strings representing the family of the font, for example, "TimesRoman", "Courier", or "Helvetica".

Font styles are constants defined by the Font class; you can get to them using class variables, for example, Font.PLAIN, Font.BOLD, or Font.ITALIC.

Finally, the point size is the size of the font, as defined by the font itself; the point size may or may not be the height of the characters.

To create an individual font object, use these three arguments to the Font class's new constructor:

```
Font f = new Font("TimesRoman", Font.BOLD, 24);
 g.setFont(f);
g.drawString(“hello”,20,40)
```

This example creates a font object for the TimesRoman bold font, in 24 points. Note that like most Java classes, you need to import the java.awt.Font class before you can use it.

The HTML APPLET Tag

The APPLET tag is used to start an applet from both an HTML document and from an applet viewer. An applet viewer will execute each APPLET tag that it finds in a separate window, while web browsers like Netscape Navigator, Internet Explorer, and HotJava will allow many applets on a single page. So far, we have been using only a simplified form of the APPLET tag. Now it is time to take a closer look at it. The syntax for the standard APPLET tag is shown here.
Bracketed items are optional.

```
< APPLET
[CODEBASE = codebaseURL]
CODE = appletFile
[ALT = alternateText]
[NAME = appletInstanceName]
WIDTH = pixels HEIGHT = pixels
[ALIGN = alignment]
[VSPACE = pixels] [HSPACE = pixels]
>
[< PARAM NAME = AttributeName VALUE = AttributeValue>]
[< PARAM NAME = AttributeName2 VALUE = AttributeValue>]
. . .
[HTML Displayed in the absence of Java]
</APPLET>
```

CODEBASE

CODEBASE is an optional attribute that specifies the base URL of the applet code, which is the directory that will be searched for the applet's executable class file (specified by the CODE tag). The HTML document's URL directory is used as the CODEBASE if this attribute is not specified. The CODEBASE does not have to be on the host from which the HTML document was read.

CODE

CODE is a required attribute that gives the name of the file containing your applet's compiled **.class** file. This file is relative to the code base URL of the applet, which is the directory that the HTML file was in or the directory indicated by CODEBASE if set.

ALT

The ALT tag is an optional attribute used to specify a short text message that should be displayed if the browser understands the APPLET tag but can't currently run Java applets. This is distinct from the alternate HTML you provide for browsers that don't support applets.

NAME

NAME is an optional attribute used to specify a name for the applet instance. Applets must be named in order for other applets on the same page to find them by name and communicate with them. To obtain an applet by name, use **getApplet( )**, which is defined by the **AppletContext** interface.

WIDTH AND HEIGHT

WIDTH and HEIGHT are required attributes that give the size (in pixels) of the applet display area.

ALIGN

ALIGN is an optional attribute that specifies the alignment of the applet. This attribute is treated the same as the HTML IMG tag with these possible values:
LEFT, RIGHT, TOP, BOTTOM, MIDDLE, BASELINE, TEXTTOP, ABSMIDDLE, and ABSBOTTOM.

VSPACE AND HSPACE

These attributes are optional. VSPACE specifies the space, in pixels, above and below the applet. HSPACE specifies the space, in pixels, on each side of the applet. They're treated the same as the IMG tag's VSPACE and HSPACE attributes.

PARAM NAME AND VALUE

The PARAM tag allows you to specify appletspecific arguments in an HTML page. Applets access their attributes with the **getParameter( )** method.

```
// Use Parameters
import java.awt.*;
import java.applet.*;
/*
<applet code="ParamDemo" width=300 height=80>
<param name=fontName value=Courier>
<param name=fontSize value=14>
<param name=leading value=2>
<param name=accountEnabled value=true>
</applet>
*/
public class ParamDemo extends Applet
{
        String fontName;
        int fontSize;
        float leading;
        boolean active;

        // Initialize the string to be displayed.
        public void start()
         {
                String param;
                fontName = getParameter("fontName");
                if(fontName == null)
                fontName = "Not Found";
                param = getParameter("fontSize");
                try
                {
                        if(param != null) // if not found
                                fontSize = Integer.parseInt(param);
                        else
                                fontSize = 0;
                } catch(NumberFormatException e) {
                        fontSize = -1;
                }
                param = getParameter("leading");
                try
```

```
            {
                if(param != null) // if not found
                    leading = Float.valueOf(param).floatValue();
                else
                    leading = 0;
            } catch(NumberFormatException e) {
                leading = -1;
            }
        param = getParameter("accountEnabled");
        if(param != null)
        active = Boolean.valueOf(param).booleanValue();
        }
// Display parameters.
public void paint(Graphics g) {
g.drawString("Font name: " + fontName, 0, 10);
g.drawString("Font size: " + fontSize, 0, 26);
g.drawString("Leading: " + leading, 0, 42);
g.drawString("Account Active: " + active, 0, 58);
}
}
```

Applet Programs

/ 1) Create an applet to draw Face. */*

```
import java.awt.*;
import java.applet.*;

/* <applet code="face" height="1000" width="1000"></applet>*/

public class face extends Applet
{
        public void paint(Graphics g)
        {
                g.drawOval(200,200,150,250);
                g.drawOval(230,270,30,20);
                g.fillOval(240,280,10,10);
                g.drawOval(290,270,30,20);
                g.fillOval(300,280,10,10);
                g.drawOval(270,325,20,40);
```

```
                g.fillArc(230,365,90,50,180,180);
        }
}
```

```
/* 2) Create an applet for displaying Banner. */
import java.awt.*;
import java.applet.*;

/* <applet code="Banner" width="1000" height="1000"> </applet> */

public class Banner extends Applet implements Runnable
{
	String msg;
	Thread t;
	public void init()
	{
		setBackground(Color.green);
		setForeground(Color.red);
		msg=" James Bond ";
	}
	public void start()
	{
		t=new Thread(this);
		t.start();
	}
	public void run()
	{
		char ch;
		while(true)
		{
			repaint();
			try
			{
				t.sleep(1000);
			}
			catch (Exception e)
			{
			}
			ch = msg.charAt(0);
			msg = msg.substring(1,msg.length());
			msg = msg+ch;
		}
	}
	public void paint(Graphics g)
```

```
        {
                g.drawString(msg,200,200);
        }
}
```

```
/* 3) Create an applet that move Ball from Left to Right. */
import java.awt.*;
import java.applet.*;
/* <applet code="Ball" Width="600" height="1000"></applet>*/
public class Ball extends Applet implements Runnable
{
	Thread t;
	int x,y;
	public void init()
	{
		x=0;
		y=10;
		t= new Thread(this);
		setBackground(Color.blue);
	}
	public void start()
	{
		t=new Thread(this);
		t.start();
	}
	public void run()
	{
		try
		{
			for(;y<=300;)
			{
				for(;x<=300;)
				{
					x+=5;
					y+=5;
					repaint();
					t.sleep(100);
				}
			y+=5;
			}
		}
		catch(Exception e)
		{ }
	}
	public void paint(Graphics g)
```

```
        {
                g.setColor(Color.cyan);
                g.fillOval(x,y,40,40);
                if(x>=300 || y>=300);
        }
}
```

```
/*4) Create an applet that will display Bouncing Ball. */
import java.awt.*;
import java.applet.*;
/* <applet code="BounceBall" width="1000" height="1000"> </applet>*/
public class BounceBall extends Applet implements Runnable
{
	int i=50, j=50, f1=0, f2=0;
	Thread t;
	public void init()
	{
		setBackground(Color.green);
	}
	public void start()
	{
		t=new Thread(this);
		t.start();
	}
	public void run()
	{
		while(true)
		{
			repaint();
			try
			{
				t.sleep(50);
			}
			catch (Exception e)
			{
			}
		}
	}
	public void paint(Graphics g)
	{
		g.setColor(Color.blue);
		g.drawRect(50,50,500,500);
		g.setColor(Color.red);
		if (j>=525)
			f1=1;
		else if (j<=50)
			f1=0;
```

```
            if (f1==1)
                j-=25;
            else
                j+=25;
            if (i>=525)
                f2=1;
            else if (i<=50)
                f2=0;
            if (f2==1)
                i-=3;
            else
                i+=3;
            g.fillOval(i,j,25,25);
        }
}

/* 5) Create an applet to display Digital Clock. */
import java.awt.*;
import java.applet.*;
import java.util.*;
/*<applet code="Clock"    height="200" width="200"></applet>*/
public class Clock extends Applet implements Runnable
{
        Thread t;
        Date d;
        public void init()
        {
            setBackground(Color.cyan);
            t=new Thread(this);
            t.start();
        }
        public void paint(Graphics g)
        {
            g.drawString(d.toString(),40,40);
        }
        public void run()
        {
            while(true)
            {
                d=new Date();
```

```
                repaint();
                try
                {
                        t.sleep(1000);
                }
                catch(Exception a)
                {
                }
        }
    }
}

/* 6) Create an applet that display content in given Font. */
import java.awt.*;
import java.applet.*;
/* <applet code="FontDemo" name="First" Height = “200” Width = “200” >
    <param name="FontName" value="Calibri" />
    <param name="FontSize" value="20" />
    <param name="FontStyle" value="3" />
    </applet>
*/
public class FontDemo extends Applet
{
    String msg = " Hello World ";
    Font F;
    String FontName;
    int FontStyle;
    int FontSize;
    public void init()
    {
        setBackground(Color.blue);
        FontName = getParameter("FontName");
        FontSize = Integer.parseInt(getParameter("FontSize"));
        FontStyle = Integer.parseInt(getParameter("FontStyle"));
        F = new Font(FontName,FontStyle,FontSize);
    }
    public void paint(Graphics G)
    {
        G.setFont(F);
        G.drawString(msg,20,20);
```

```
        }
}
```